Face Attendance API — Endpoint Reference for App Developers

Base URL `http://localhost:5080` · Generated 9 October 2026 from `docs/FRONTEND-API.md`

This document lists **every endpoint the app uses**. Each endpoint is described in the same format: method, URL, headers (with the exact value to enter), path / query / body parameters, a full request example, and the real response.

Every request and response below was run against a live local instance.

Quick reference

#	What it does	Method	URL	API key
0	Check the server is up	`GET`	`http://localhost:5080/health`	none
1	Register a person with 3 photos	`POST`	`http://localhost:5080/api/v1/registrations`	admin
2	Find a person by employee ID	`GET`	`http://localhost:5080/api/v1/persons/by-external-id/{externalId}`	admin
3	Check registration status	`GET`	`http://localhost:5080/api/v1/persons/{personId}/enrollment`	admin
4	Mark attendance	`POST`	`http://localhost:5080/api/v1/attendance/mark`	device
5	Attendance list / report	`GET`	`http://localhost:5080/api/v1/attendance`	admin

Before you start

Base URL

```
http://localhost:5080
```

Start the API first: press **F5** in VS Code, or run `dotnet run --project src/Api` from the project folder. Open `http://localhost:5080/health` in a browser — it should say `Healthy`.

> **If Postman never gets a response:** the app was started with the debugger and has paused on an error. In VS Code press **F5 (Continue)**, or **Shift+F5** to stop, then start it with the default configuration **"Run API (for Postman / app testing)"**, which runs without the debugger. Under the debugger, every rejected request (409, 404, 422...) freezes the whole app.

API keys — what to put in the `X-Api-Key` header

Every endpoint except `/health` needs a header named `X-Api-Key`. Locally, use these values exactly:

Key type	Value to enter locally	Use it for
Admin key	`dev-admin-key`	Endpoints 1, 2, 3, 5 (registration screen, reports)
Device key	`dev-device-key`	Endpoint 4 only (attendance kiosk / mobile app)

Where these come from: nothing generates or issues them. They are plain text values written in the API's settings file `src/Api/appsettings.Development.json`:

```
"ApiKeys": {
  "Admin":  [ "dev-admin-key" ],
  "Device": [ "dev-device-key" ]
}
```

The API compares your `X-Api-Key` header against this list. To use your own keys, change the values in that file (or set environment variables `ApiKeys__Admin__0` and `ApiKeys__Device__0`) and **restart the API**. On a production server the list starts empty, so every request is refused until real keys are configured there.

Adding your own key locally — two mistakes that silently fail (both return `401`):

Wrong	Why it fails	Right
`"Admin": "my-key"`	Must be a list in square brackets. A plain string is ignored	`"Admin": [ "my-key" ]`
Putting the key in `appsettings.json`	When running locally, `appsettings.Development.json` replaces the first key in that list with `dev-admin-key`	Put it in `appsettings.Development.json`

Working example — keeps the default key and adds yours, in `src/Api/appsettings.Development.json`:

```
"ApiKeys": {
  "Admin":  [ "dev-admin-key", "my-own-admin-key" ],
  "Device": [ "dev-device-key" ]
}
```

If you send	You get
No `X-Api-Key` header, or a wrong value	`401 Unauthorized` (empty body)
A device key on an admin endpoint	`403 Forbidden` (empty body)
An admin key on endpoint 4	Allowed — the admin key works everywhere

> **Security:** never put the admin key inside a browser page or a mobile app — anyone can read it there. A web registration screen should call your own backend, which adds the admin key. The device key can live in a kiosk or mobile app because it can only mark attendance.

IDs — which are created automatically and which you send

All database IDs are created automatically by SQL Server. They are short whole numbers that start at **10001** and go up by 1 for each new record: `10001`, `10002`, `10003` …

Each table counts on its own, so the first person, the first photo and the first attendance record can all be `10001`.

You never send an ID when creating something. If a request includes an `id` field anyway, it is ignored — this was tested: sending `"id": 5` produced `10003`, the next number in sequence.

Field	What it is	Example	Who creates it	Do you send it?
`person.id` (also called `personId`)	The person's database ID	`10001`	**SQL Server, automatically**, when you call endpoint 1	Never when creating. You send it **back** later in the URL of endpoint 3, to say *which* person
`templateId`	ID of each stored photo	`10001`	**SQL Server, automatically**	Never
`eventId`	ID of each attendance attempt	`10001`	**SQL Server, automatically**, when you call endpoint 4	Never
`externalId`	**Your own** employee / student code	`EMP-1001`	**You**	**Yes**, on endpoint 1. It is not a database ID — it is your code from your own HR system, so both systems can refer to the same person. Must be unique

Three things to know about these numbers:

- **They are numbers in JSON, not text:** `"id": 10001`, not `"id": "10001"`. Store them as numbers (`int`) in the app.
- **There can be gaps.** A registration that fails still uses up a number, and SQL Server may skip ahead after a restart. `10001, 10002, 10004` is normal. Never assume the next ID.
- **`personId` and `externalId` are different things.** `10001` is the database's number; `EMP-1001` is yours. Endpoint 3 needs the number.

Two rules that apply to every endpoint

1. **Never add a `Content-Type` header yourself** on endpoints that upload photos (1 and 4). Postman, browsers and `fetch` add `multipart/form-data` with the correct boundary automatically. Adding it by hand breaks the upload.
2. **Only `X-Api-Key` is needed.** No other header is required. (`X-Tenant-Id` exists only for multi-site deployments — leave it out.)

0. Check the server is up

What it does: confirms the API and database are running. **Used by:** app start-up check.

Item	Value
Method	`GET`
URL	`http://localhost:5080/health`
Body	none

Headers

None. This is the only endpoint that needs no API key.

Parameters

None.

Example request

```
curl http://localhost:5080/health
```

Response — `200 OK`

Plain text, not JSON:

```
Healthy
```

If the database is down it returns `503` with the text `Unhealthy`.

1. Register a person with 3 photos

What it does: creates the person and stores their front, left and right face photos in one call. **Used by:** the registration screen.

Item	Value
Method	`POST`
URL	`http://localhost:5080/api/v1/registrations`
Body format	`form-data` (multipart)

Headers

Header name	Value to enter	Required
`X-Api-Key`	`dev-admin-key`	**Yes**

Do **not** add `Content-Type`.

Path parameters

None.

Query parameters

None.

Body — `form-data`

Key (exact spelling)	Type	Required	Example value	Rules
`externalId`	Text	**Yes**	`EMP-1001`	Your employee code. Max 64 characters. Must not already be registered
`fullName`	Text	**Yes**	`Alice Kumar`	Max 200 characters
`email`	Text	No	`alice.kumar@company.com`	Must be a valid email if sent
`department`	Text	No	`Operations`	Max 128 characters
`front`	**File**	**Yes**	`alice-front.jpg`	Person looking straight at the camera
`left`	**File**	**Yes**	`alice-left.jpg`	Head turned to the **person's own** left
`right`	**File**	**Yes**	`alice-right.jpg`	Head turned to the **person's own** right

Photo rules (each file): JPEG, PNG or WebP · max 8 MB · at least 240 × 240 pixels · exactly one face in the photo · the three photos must be different pictures.

Do not send `id` or `personId` — the API creates the ID.

Example request — curl

Run from the `docs/samples` folder:

```
curl -X POST http://localhost:5080/api/v1/registrations \
  -H "X-Api-Key: dev-admin-key" \
  -F "externalId=EMP-1001" \
  -F "fullName=Alice Kumar" \
  -F "email=alice.kumar@company.com" \
  -F "department=Operations" \
  -F "front=@alice-front.jpg" \
  -F "left=@alice-left.jpg" \
  -F "right=@alice-right.jpg"
```

Example request — Postman

1. Method **POST**, URL `http://localhost:5080/api/v1/registrations`
2. **Headers** tab → add one row: Key `X-Api-Key`, Value `dev-admin-key`
3. **Body** tab → choose **form-data** → add these rows:

Key	Type (dropdown on the key cell)	Value
`externalId`	Text	`EMP-1001`
`fullName`	Text	`Alice Kumar`
`email`	Text	`alice.kumar@company.com`
`department`	Text	`Operations`
`front`	**File**	select `alice-front.jpg`
`left`	**File**	select `alice-left.jpg`
`right`	**File**	select `alice-right.jpg`

Success response — `201 Created`

The fields the app needs:

Field	Example	Meaning
`person.id`	`10001`	**Save this.** The ID the API created for this person
`person.externalId`	`EMP-1001`	The code you sent
`enrollmentStatus`	`Enrolled`	`Enrolled` = ready to mark attendance
`enrollmentCompleted`	`true`	`true` = all 3 photos accepted

```
{
  "person": {
    "id": 10001,
    "externalId": "EMP-1001",
    "fullName": "Alice Kumar",
    "email": "alice.kumar@company.com",
```

```
    "department": "Operations",
    "status": "Active",
    "enrollmentStatus": "Enrolled",
    "enrolledAtUtc": "2026-10-09T05:53:54.5368845+00:00",
    "createdAtUtc": "2026-10-09T05:53:54.1089243+00:00"
  },
  "captures": [ "...one object per photo, see full response below..." ],
  "enrollmentStatus": "Enrolled",
  "enrollmentCompleted": true
}
```


Full response, exactly as returned


```
{
  "person": {
    "id": 10001,
    "externalId": "EMP-1001",
    "fullName": "Alice Kumar",
    "email": "alice.kumar@company.com",
    "department": "Operations",
    "status": "Active",
    "enrollmentStatus": "Enrolled",
    "enrolledAtUtc": "2026-10-09T05:53:54.5368845+00:00",
    "createdAtUtc": "2026-10-09T05:53:54.1089243+00:00"
  },
  "captures": [
    {
      "personId": 10001,
      "pose": "Front",
      "templateId": 10001,
      "version": 1,
      "replacedPrevious": false,
      "enrollmentStatus": "InProgress",
      "capturedPoses": [ "Front" ],
      "missingPoses": [ "Left", "Right" ],
      "enrollmentCompleted": false,
      "analysis": {
        "faceDetected": true,
        "faceCount": 1,
        "detectionScore": 0.99,
        "qualityScore": 0.9,
        "sharpness": 0.9,
        "brightness": 0.5,
        "imageWidth": 640,
        "imageHeight": 640,
        "yawDegrees": 0,
        "pitchDegrees": 0,
        "rollDegrees": 0,
        "inferredPose": "Front",
```

```
      "box": { "x": 100, "y": 100, "width": 200, "height": 200 },
      "warnings": [ "Development provider: no real face analysis was performed." ]
    },
    "warnings": [ "Development provider: no real face analysis was performed." ]
  },
  {
    "personId": 10001,
    "pose": "Left",
    "templateId": 10002,
    "version": 1,
    "replacedPrevious": false,
    "enrollmentStatus": "InProgress",
    "capturedPoses": [ "Front", "Left" ],
    "missingPoses": [ "Right" ],
    "enrollmentCompleted": false,
    "analysis": {
      "faceDetected": true,
      "faceCount": 1,
      "detectionScore": 0.99,
      "qualityScore": 0.9,
      "sharpness": 0.9,
      "brightness": 0.5,
      "imageWidth": 640,
      "imageHeight": 640,
      "yawDegrees": 22,
      "pitchDegrees": 0,
      "rollDegrees": 0,
      "inferredPose": "Left",
      "box": { "x": 100, "y": 100, "width": 200, "height": 200 },
      "warnings": [ "Development provider: no real face analysis was performed." ]
    },
    "warnings": [ "Development provider: no real face analysis was performed." ]
  },
  {
    "personId": 10001,
    "pose": "Right",
    "templateId": 10003,
    "version": 1,
    "replacedPrevious": false,
    "enrollmentStatus": "Enrolled",
    "capturedPoses": [ "Front", "Left", "Right" ],
    "missingPoses": [],
    "enrollmentCompleted": true,
    "analysis": {
      "faceDetected": true,
      "faceCount": 1,
      "detectionScore": 0.99,
      "qualityScore": 0.9,
      "sharpness": 0.9,
```

```
        "brightness": 0.5,
        "imageWidth": 640,
        "imageHeight": 640,
        "yawDegrees": -22,
        "pitchDegrees": 0,
        "rollDegrees": 0,
        "inferredPose": "Right",
        "box": { "x": 100, "y": 100, "width": 200, "height": 200 },
        "warnings": [ "Development provider: no real face analysis was performed." ]
      },
      "warnings": [ "Development provider: no real face analysis was performed." ]
    }
  ],
  "enrollmentStatus": "Enrolled",
  "enrollmentCompleted": true
}
```


The `warnings` saying "Development provider" appear only in local test mode.

All or nothing: if any of the three photos is rejected, the person is **not** saved, so you can fix the photo and send the same request again with the same `externalId`.

Error responses

Status	`code`	When
`400`	—	A required field is missing (see `errors`)
`401`	—	`X-Api-Key` missing or wrong
`403`	—	Device key used instead of admin key
`409`	`external_id_exists`	This `externalId` is already registered
`409`	`duplicate_identity`	This **face** is already registered under a different `externalId`
`409`	`duplicate_frame`	The same picture was sent for two of the three photos
`413`	`image_too_large`	A photo is over 8 MB
`415`	`unsupported_format`	A file is not a real JPEG / PNG / WebP
`422`	`no_face_detected`	No face found in a photo
`422`	`multiple_faces_detected`	More than one person in a photo
`422`	`low_quality`	Photo blurry, too dark, or face too small — read `hints`
`422`	`pose_mismatch`	Head not turned the way that photo requires

Missing field — `400`:


```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
```

```
  "errors": {
    "Left": [ "The Left field is required." ],
    "Right": [ "The Right field is required." ]
  },
  "traceId": "00-78a063054f04234dfac9920de2435ab2-11ca7208035eb8c7-00"
}
```

Employee code already registered — `409`:

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.10",
  "title": "The request could not be completed",
  "status": 409,
  "detail": "A person with ExternalId EMP-1001 already exists in this tenant.",
  "code": "external_id_exists",
  "hints": [],
  "traceId": "00-339d69d8e3baadf79e150710344e3b50-3218ab42e49a68d0-00"
}
```

Same face already registered under another code — `409`:

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.10",
  "title": "The image could not be used",
  "status": 409,
  "detail": "This face already matches an existing registration (similarity 1.00).",
  "code": "duplicate_identity",
  "hints": [
    "If this person is genuinely new, an operator should review the existing record.",
    "Conflicting person id: 10001"
  ],
  "traceId": "00-a56af66bfb2777b24d8da9044ae53c5f-a0b6592fb10b43c9-00"
}
```

2. Find a person by employee ID

What it does: returns a person using **your** employee code — useful because the app usually knows `EMP-1001`, not the database number `10001`. **Used by:** registration screen, admin screens.

Item	Value
Method	`GET`
URL	`http://localhost:5080/api/v1/persons/by-external-id/{externalId}`
Body	none

Headers

Header name	Value to enter	Required
`X-Api-Key`	`dev-admin-key`	**Yes**

Path parameters

Parameter	Required	Example value	Meaning
`externalId`	**Yes**	`EMP-1001`	Your employee code, exactly as sent at registration

Query parameters

None.

Body

None.

Example request

```
curl http://localhost:5080/api/v1/persons/by-external-id/EMP-1001 \
  -H "X-Api-Key: dev-admin-key"
```

Postman: Method **GET** · URL `http://localhost:5080/api/v1/persons/by-external-id/EMP-1001` · **Headers** tab: `X-Api-Key` = `dev-admin-key` · no body.

Success response — `200 OK`

```
{
  "id": 10001,
  "externalId": "EMP-1001",
  "fullName": "Alice Kumar",
  "email": "alice.kumar@company.com",
  "department": "Operations",
  "status": "Active",
  "enrollmentStatus": "Enrolled",
  "enrolledAtUtc": "2026-10-09T05:53:54.5368845+00:00",
```

```
  "createdAtUtc": "2026-10-09T05:53:54.1089243+00:00"
}
```


Field	Meaning
`id`	The `personId` — a number such as `10001` (needed for endpoint 3)
`status`	`Active`, `Suspended` or `Archived`
`enrollmentStatus`	`Enrolled` = ready for attendance. See endpoint 3 for all values

Error responses

Status	`code`	When
`401`	—	`X-Api-Key` missing or wrong
`403`	—	Device key used
`404`	`person_not_found`	No person with that code


```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.5",
  "title": "The request could not be completed",
  "status": 404,
  "detail": "No person with ExternalId EMP-9999 in this tenant.",
  "code": "person_not_found",
  "hints": [],
  "traceId": "00-bcc67b48dc4312603a2714ade7b14876-2bfbaeddf55362de-00"
}
```

3. Check registration status

What it does: shows which of the 3 photos are stored and whether the person can mark attendance. **Used by:** registration screen (after registering, or to resume a half-done one).

Item	Value
Method	`GET`
URL	`http://localhost:5080/api/v1/persons/{personId}/enrollment`
Body	none

Headers

Header name	Value to enter	Required
`X-Api-Key`	`dev-admin-key`	**Yes**

Path parameters

Parameter	Required	Example value	Where to get it
`personId`	**Yes**	`10001`	`person.id` from endpoint 1, or `id` from endpoint 2. A number such as `10001` — `EMP-1001` will not work here

Query parameters

None.

Body

None.

Example request

```
curl http://localhost:5080/api/v1/persons/10001/enrollment \
  -H "X-Api-Key: dev-admin-key"
```

Postman: Method **GET** · URL `http://localhost:5080/api/v1/persons/10001/enrollment` (replace `10001` with your person ID) · **Headers** tab: `X-Api-Key` = `dev-admin-key` · no body.

Success response — `200 OK`


```
{
  "personId": 10001,
  "externalId": "EMP-1001",
  "fullName": "Alice Kumar",
  "status": "Active",
  "enrollmentStatus": "Enrolled",
  "enrolledAtUtc": "2026-10-09T05:53:54.5368845+00:00",
  "poses": [
    {
```

```
      "pose": "Front",
      "templateId": 10001,
      "version": 1,
      "qualityScore": 0.9,
      "detectionScore": 0.99,
      "providerId": "development:filename-stub:v1",
      "providerMatchesCurrent": true,
      "capturedAtUtc": "2026-10-09T05:53:54.3965763+00:00"
    },
    {
      "pose": "Left",
      "templateId": 10002,
      "version": 1,
      "qualityScore": 0.9,
      "detectionScore": 0.99,
      "providerId": "development:filename-stub:v1",
      "providerMatchesCurrent": true,
      "capturedAtUtc": "2026-10-09T05:53:54.5137154+00:00"
    },
    {
      "pose": "Right",
      "templateId": 10003,
      "version": 1,
      "qualityScore": 0.9,
      "detectionScore": 0.99,
      "providerId": "development:filename-stub:v1",
      "providerMatchesCurrent": true,
      "capturedAtUtc": "2026-10-09T05:53:54.5247612+00:00"
    }
  ],
  "missingPoses": [],
  "poseSimilarities": [
    { "a": "Front", "b": "Left", "similarity": 0.9858 },
    { "a": "Front", "b": "Right", "similarity": 0.985 },
    { "a": "Left", "b": "Right", "similarity": 0.9862 }
  ],
  "warnings": []
}
```


`enrollmentStatus`	Meaning	What the app should do
`Enrolled`	All 3 photos stored and checked	Person can mark attendance
`InProgress`	Some photos missing	Ask for the poses listed in `missingPoses`
`NotEnrolled`	No photos yet	Start registration
`Stale`	The face engine was changed since these photos were taken	Register the person again

Error responses

Status	`code`	When
`401`	—	`X-Api-Key` missing or wrong
`403`	—	Device key used
`404`	`person_not_found`	No person with that ID
`404`	— (empty body)	`personId` is not a number

4. Mark attendance

What it does: checks the live photo against the registered photos and records the punch. **Used by:** the attendance kiosk / mobile app.

Item	Value
Method	`POST`
URL	`http://localhost:5080/api/v1/attendance/mark`
Body format	`form-data` (multipart)

Headers

Header name	Value to enter	Required
`X-Api-Key`	`dev-device-key`	**Yes** (`dev-admin-key` also works)

Do **not** add `Content-Type`.

Path parameters

None.

Query parameters

None.

Body — `form-data`

Key (exact spelling)	Type	Required	Example value	Rules
`image`	**File**	**Yes**	`alice-front.jpg`	The live camera photo. Same photo rules as registration
`externalId`	Text	**Recommended**	`EMP-1001`	Who is claiming to be here (from a badge, PIN or name picker). Makes the check much more accurate
`personId`	Text	No	`10001`	Alternative to `externalId`. If both are sent, `personId` is used
`deviceId`	Text	No	`GATE-A`	Which kiosk sent it. Shows up in reports. Max 128 characters
`latitude`	Number	No	`19.0760`	−90 to 90
`longitude`	Number	No	`72.8777`	−180 to 180
`capturedAtUtc`	Text	No	`2026-10-09T05:53:55Z`	When the device took the photo (ISO 8601). Stored for reference only
`direction`	Text	No	`In`	`In` or `Out`. **Normally leave it out** — the server decides In/Out from the person's previous punch, and ignores this field once the person has punched before

With `externalId` (recommended): the photo is compared only with that person — fast and accurate. **Without `externalId` or `personId`:** the photo is compared with **everyone** registered. It works, but becomes less reliable as the number of people grows.

Do not send `id` or `eventId` — the API creates them.

Example request — curl

```
curl -X POST http://localhost:5080/api/v1/attendance/mark \
  -H "X-Api-Key: dev-device-key" \
  -F "externalId=EMP-1001" \
  -F "deviceId=GATE-A" \
  -F "image=@alice-front.jpg"
```

Example request — Postman

1. Method **POST**, URL `http://localhost:5080/api/v1/attendance/mark`
2. **Headers** tab → Key `X-Api-Key`, Value `dev-device-key`
3. **Body** tab → **form-data** →

Key	Type	Value
`externalId`	Text	`EMP-1001`
`deviceId`	Text	`GATE-A`
`image`	**File**	select `alice-front.jpg`

Response — always `200 OK` when the request was valid

Important: even when the face does **not** match, the status is `200`. Always read `success` and `decision` in the body.

Accepted:

```
{
  "success": true,
  "decision": "Accepted",
  "eventId": 10001,
  "personId": 10001,
  "externalId": "EMP-1001",
  "fullName": "Alice Kumar",
  "direction": "In",
  "score": 1,
  "threshold": 0.42,
  "matchedPose": "Left",
  "recordedAtUtc": "2026-10-09T05:53:55.1068887+00:00",
  "processingMs": 86,
  "topScores": [
    { "personId": 10001, "pose": "Left", "score": 1 }
  ]
}
```

Same person again within 2 minutes:


```
{
  "success": false,
  "decision": "Duplicate",
  "eventId": 10002,
  "personId": 10001,
  "externalId": "EMP-1001",
  "fullName": "Alice Kumar",
  "score": 1,
  "threshold": 0.42,
  "matchedPose": "Left",
  "recordedAtUtc": "2026-10-09T05:53:55.3553255+00:00",
  "message": "Already marked In at 2026-10-09T05:53:55.1068887+00:00, inside the 2-minute window.",
  "processingMs": 8,
  "topScores": [
    { "personId": 10001, "pose": "Left", "score": 1 }
  ]
}
```


Face does not match the claimed person:


```
{
  "success": false,
  "decision": "Rejected",
  "eventId": 10003,
  "personId": 10001,
  "externalId": "EMP-1001",
  "fullName": "Alice Kumar",
  "score": -0.0101,
  "threshold": 0.42,
  "recordedAtUtc": "2026-10-09T05:53:55.5133388+00:00",
  "message": "Best score -0.010 did not reach the 0.420 threshold.",
  "processingMs": 3,
  "topScores": [
    { "personId": 10001, "pose": "Left", "score": -0.0101 }
  ]
}
```


Response fields

Field	Meaning
`success`	`true` only when attendance was recorded as `Accepted`
`decision`	The result — see table below
`eventId`	ID of this attempt, created by the API. Every attempt is saved, even failed ones
`personId`, `externalId`, `fullName`	The matched (or claimed) person. Missing when nobody matched
`direction`	`In` or `Out` — only present when `Accepted`

Field	Meaning
`score` , `threshold`	Match score and the minimum needed. **Do not show these to end users**
`matchedPose`	Which stored photo (`Front` / `Left` / `Right`) matched best
`message`	Reason, when `success` is `false`
`recordedAtUtc`	Server time of the punch (UTC)
`processingMs`	Time the server took
`topScores`	Debug information, returned only when running locally. Empty on a real server

What to show for each `decision`

`decision`	`success`	Message to show the user
`Accepted`	`true`	"Welcome, {fullName} — marked {direction}"
`Duplicate`	`false`	"Already marked a moment ago"
`Rejected`	`false`	"Face not recognised, please try again"
`NotEnrolled`	`false`	"You are not registered yet"
`NoFaceDetected`	`false`	"Please look at the camera"
`LowQuality`	`false`	"Too dark or blurry, please try again"
`ProviderError`	`false`	"Something went wrong, please contact admin"

Error responses (request itself was wrong)

Status	`code`	When
`400`	—	`image` missing
`401`	—	`X-Api-Key` missing or wrong
`404`	`person_not_found`	The `externalId` / `personId` sent does not exist
`413`	`image_too_large`	Photo over 8 MB
`415`	`unsupported_format`	File is not a real JPEG / PNG / WebP
`429`	—	More than 60 requests per minute from one device

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.16",
  "title": "The image could not be used",
  "status": 415,
  "detail": "The file content is not a JPEG, PNG or WebP image, whatever the content type says.",
  "code": "unsupported_format",
  "hints": [],
  "traceId": "00-f41a133d0f2fea47ee9adef538a81ccd-6f5f300015a31ceb-00"
}
```

5. Attendance list / report

What it does: returns attendance records, newest first, with filters. **Used by:** reports, admin dashboard.

Item	Value
Method	`GET`
URL	`http://localhost:5080/api/v1/attendance`
Body	none

Headers

Header name	Value to enter	Required
`X-Api-Key`	`dev-admin-key`	**Yes**

Path parameters

None.

Query parameters

All optional. Add them after `?`, joined with `&`.

Parameter	Example value	Meaning
`decision`	`Accepted`	Only this result. Use `Accepted` for the real attendance register. Other values: `Rejected`, `Duplicate`, `NotEnrolled`, `NoFaceDetected`, `LowQuality`, `ProviderError`
`personId`	`10001`	Only one person
`fromUtc`	`2026-10-09T00:00:00Z`	From this time (included). Use the `Z` (UTC) format
`toUtc`	`2026-10-10T00:00:00Z`	Up to this time (not included)
`page`	`1`	Page number. Default `1`
`pageSize`	`50`	Records per page. Default `50`, maximum `500`

Without `decision`, failed attempts are included too — useful when someone says "the machine didn't recognise me".

Body

None.

Example request — today's attendance

```
curl "http://localhost:5080/api/v1/attendance?decision=Accepted&fromUtc=2026-10-09T00:00:00Z&toUtc=2026-10-10T00:00:00Z&page=1&pageSize=50" \
  -H "X-Api-Key: dev-admin-key"
```

Postman: Method **GET** · URL `http://localhost:5080/api/v1/attendance` · **Params** tab add rows `decision` = `Accepted`, `fromUtc` = `2026-10-09T00:00:00Z`, `toUtc` = `2026-10-10T00:00:00Z`, `page` = `1`, `pageSize` = `50` · **Headers** tab: `X-Api-Key` = `dev-admin-key` · no body.

Success response — `200 OK`


```
{
  "items": [
    {
      "id": 10001,
      "personId": 10001,
      "externalId": "EMP-1001",
      "fullName": "Alice Kumar",
      "direction": "In",
      "decision": "Accepted",
      "matchScore": 1,
      "threshold": 0.42,
      "matchedPose": "Left",
      "deviceId": "GATE-A",
      "createdAtUtc": "2026-10-09T05:53:55.1068887+00:00",
      "processingMs": 86
    }
  ],
  "page": 1,
  "pageSize": 50,
  "total": 1,
  "totalPages": 1
}
```


Field	Meaning
`items[].id`	The `eventId` of the punch
`items[].createdAtUtc`	When the punch was recorded (UTC) — convert to local time in the app
`items[].failureReason`	Present only on failed attempts
`total`	Total matching records across all pages
`totalPages`	Number of pages at this `pageSize`

Error responses

Status	When
`401`	`X-Api-Key` missing or wrong
`403`	Device key used

Error format (all endpoints)

Errors come in two shapes. **Check the `code` field in your app, never the `detail` text.**

Missing or invalid fields → `400` with `errors` listing each field:

```
{
  "status": 400,
  "title": "One or more validation errors occurred.",
  "errors": { "Left": [ "The Left field is required." ] }
}
```

Everything else → `code`, `detail` **and** `hints`**:**

```
{
  "status": 409,
  "detail": "A person with ExternalId EMP-1001 already exists in this tenant.",
  "code": "external_id_exists",
  "hints": []
}
```

`401` **and** `403` **have an empty body.**

Local testing note

Locally, the API uses a **test face engine that matches by file name, not by the actual face**. `alice-front.jpg`, `alice-left.jpg` and `alice-right.jpg` are treated as the same person; `bob-front.jpg` is a different person. The images in `docs/samples` are placeholders, not real faces.

So locally you can test every request, response and error in this document. Real face matching starts once the face models are installed; the **URLs, headers, fields and responses stay exactly the same**.

All of these endpoints are also listed interactively at `http://localhost:5080/scalar/v1`, and a ready-made Postman collection is in `FaceAttendance.postman_collection.json`.